

Benemérita Universidad Autónoma de Puebla

Ingeniería en Ciencia de Datos

Programación Avanzada

Carlos Alexander Castillo Gomez

Prof. Jaime Alejandro Romero Sierra

Actividad: Ejercicios con diagramas UML

Primavera 2025

Fecha de entrega: 11/02/2025

Justificación e Importancia del Sistema de reservas cine

(cine)

Justificación del Sistema

En la actualidad, los sistemas de gestión de cines se han convertido en una herramienta fundamental para la industria del entretenimiento. Este sistema ha sido diseñado para abordar diversos aspectos críticos que enfrentan los complejos cinematográficos modernos:

1. Eficiencia Operativa.

- Automatización de la gestión de salas y funciones
- Optimización en la venta y distribución de boletos
- Control efectivo de la ocupación de salas
- Gestión eficiente del personal y recursos

2. Experiencia del Usuario:

- Facilidad para realizar reservas múltiples
- Sistema transparente de disponibilidad de asientos
- Acceso a información detallada de películas
- Gestión personalizada de preferencias
- Sistema de promociones y descuentos

3. Control Administrativo:

- Seguimiento en tiempo real de la ocupación
- Gestión de empleados y roles
- Administración de promociones y precios
- Control de servicios adicionales (zona de comidas)

Impacto en la Sociedad

El sistema tiene un impacto significativo en diferentes aspectos:

1. Para los Espectadores:

- Reservas más rápidas y seguras
- Mejor planificación de su entretenimiento
- Acceso a promociones y descuentos
- Experiencia más organizada y satisfactoria

2. Para los Empleados:

- Herramientas eficientes para gestión de salas
- Control efectivo de reservas y ocupación
- Mejor organización del trabajo
- Gestión simplificada de servicios adicionales

3. Para los Administradores:

- Control completo de operaciones
- Información en tiempo real
- Gestión eficiente de recursos
- Optimización de ingresos

Análisis Detallado de Clases

1. Clase “Usuario” (Clase Base)

Atributos:

- username: str
- contrasenia: str
- email: str

Métodos:

- registrar()

Relaciones:

- Es clase base para `Persona`

2. Clase “Persona” (Hereda de `Usuario`)

Atributos:

- nombre: str
- telefon: str
- reservas: List[Reserva]

Métodos:

- modificar_usuario()
- crear_reserva_multiple(asientos: List[int], funcion: Funcion)

Relaciones:

- Hereda de `Usuario`
- Se relaciona con `Reserva` y `Funcion`

3. Clase “Sala”

Atributos:

- aforo: int
- disponibilidad: bool
- asientos: Dict[int, bool]

Métodos:

- comprobar_disponibilidad()
- get_asientos_disponibles()
- actualizar_asientos(asientos: List[int], estado: bool)

Relaciones:

- Utilizada por `Funcion`
- Gestionada indirectamente por `Reserva`

4. Clase “Película”

Atributos:

- titulo: str
- sinopsis: str
- duracion: int
- activa: bool

Métodos:

- detalles()

Relaciones:

- Utilizada por `Funcion`
- Gestionada por `Empleado`

5. Clase “Empleado”

Atributos:

- num_registro: str
- puesto: str
- fecha_ingreso: str

Métodos:

- agregar_funcion(funcion: Funcion)
- agregar_pelicula(pelicula: Pelicula)
- agregar_promocion(promocion: Promocion)
- modificar_estado_pelicula(pelicula: Pelicula, estado: bool)

Relaciones:

- Gestiona `Funcion`, `Pelicula`, y `Promocion`
- Es clase base para `Zona\_Comida`

6. Clase “Zona_Comida” (Hereda de `Empleado`)

Atributos:

- productos: List[str]
- precios: Dict[str, float]

Métodos:

- menu()

Relaciones:

- Hereda de `Empleado`

7. Clase “Promoción”

Atributos:

- descuento: float
- beneficios: str

- activa: bool

Métodos:

- aplicar_promocion(precio: float)

Relaciones:

- Gestionada por `Empleado`
- Aplicable a `Reserva`

8. Clase “Función”

Atributos:

- hora: datetime
- fecha: date
- precio: float
- sala: Sala
- pelicula: Pelicula
- disponible: bool

Métodos:

- consultar_asiento(numero: int)
- verificar_disponibilidad_multiple(asientos: List[int])

Relaciones:

- Contiene `Sala` y `Pelicula`
- Utilizada por `Reserva`
- Gestionada por `Empleado`

9. Clase “Reserva”

Atributos:

- num_asientos: List[int]
- fecha: date
- estado: str
- funcion: Funcion
- codigo_reserva: str
- precio_total: float

Métodos:

- cancelar_reserva()
- confirmar_reserva()
- calcular_precio_total()

Relaciones:

- Se relaciona con `Funcion`
- Pertenece a `Persona`
- Puede tener `Promocion` aplicada

Conclusión

Este sistema representa una solución integral para la gestión moderna de cines, abordando tanto las necesidades operativas como la experiencia del usuario. Su diseño modular y las relaciones bien definidas entre clases permiten una gestión eficiente y escalable de todas las operaciones relacionadas con la exhibición cinematográfica, desde la venta de boletos hasta la gestión de servicios adicionales.

El sistema está diseñado para evolucionar con las necesidades cambiantes de la industria del entretenimiento, proporcionando una base sólida para la implementación de nuevas funcionalidades y la adaptación a futuros problemas.

Captura del funcionamiento del sistema

```
=== SISTEMA DE CINE ===

1. Creando salas...

2. Registrando películas...

Detalles de películas registradas:
- Rapidos y Furiosos: 148 minutos
- Me before you: 175 minutos
- La purga: 169 minutos

3. Registrando empleados...

Menú de comida disponible:
- Palomitas: $5.99
- Nachos: $6.99
- Refrescos: $3.99
- Hot Dogs: $4.99

4. Programando funciones...
Función agregada: Rapidos y Furiosos - Sala: 50 asientos
Función agregada: Me before you - Sala: 100 asientos
Función agregada: La purga - Sala: 75 asientos
...
10. Gestionando estado de películas...
Película 'Me before you' desactivada

=== FIN ===
```

Justificación e Importancia del Sistema de gestión de pedidos

(cafetería/restaurante)

Justificación del Sistema

En la era digital actual, la implementación de sistemas de gestión en restaurantes ha dejado de ser un lujo para convertirse en una necesidad fundamental. Este sistema ha sido diseñado para abordar varios desafíos críticos que enfrentan los establecimientos de servicios de alimentos y bebidas:

1. ***Eficiencia Operativa:*** La automatización de procesos reduce significativamente el tiempo dedicado a tareas administrativas, permitiendo que el personal se centre en proporcionar un servicio de calidad al cliente.
2. ***Experiencia del Cliente:*** El sistema permite una personalización detallada de productos, gestión de preferencias y un programa de fidelización que mejora significativamente la satisfacción del cliente.
3. ***Control de Inventario:*** La gestión eficiente del inventario es crucial para mantener la rentabilidad del negocio. El sistema proporciona alertas de stock bajo y seguimiento en tiempo real, reduciendo el desperdicio y optimizando los costos.
4. ***Seguridad Alimentaria:*** El contenido de ingredientes y la gestión de alérgenos son aspectos críticos para la seguridad alimentaria.

Impacto en la Sociedad

El sistema tiene un impacto significativo en diversos aspectos de nuestra vida cotidiana:

1. Para los Consumidores:

- Mayor transparencia en la información nutricional
- Personalización de productos según preferencias
- Programa de fidelización que recompensa la lealtad
- Servicio más rápido y preciso
- Mayor seguridad alimentaria

2. Para los Empleados:

- Reducción de errores en la toma de pedidos
- Mejor organización del trabajo
- Claridad en las responsabilidades
- Herramientas eficientes para realizar su trabajo

3. Para los Propietarios:

- Mejor control de costos
- Optimización de inventario
- Datos para toma de decisiones
- Incremento en la eficiencia operativa

Análisis Detallado de Clases

1. Clase “Persona” (Clase Base)

Atributos:

- nombre: str
- identificacion: str
- telefono: str

Métodos:

- actualizar_datos(nombre: str, telefono: str)

Relaciones:

- Es clase base para `Cliente` y `Empleado`

2. Clase “Cliente” (Hereda de `Persona`)

Atributos:

- historial_pedidos: List[Pedido]
- puntos_fidelidad: int

Métodos:

- realizar_pedido(pedido: Pedido)
- consultar_historial()
- obtener_puntos()

Relaciones:

- Hereda de `Persona`
- Se relaciona con `Pedido` (composición)

3. Clase “Empleado” (Hereda de `Persona`)

Atributos:

- rol: str
- horario: str
- salario: float

Métodos:

- actualizar_inventario(inventario: Inventario, producto: str, cantidad: int)
- gestionar_pedidos(pedido: Pedido, nuevo_estado: EstadoPedido)
- aplicar_promociones(pedido: Pedido, promocion: Promocion)

Relaciones:

- Hereda de `Persona`
- Interactúa con `Inventario`, `Pedido` y `Promocion`

4. Clase “ProductoBase” (Clase Base)

Atributos:

- nombre: str
- precio: float
- descripcion: str
- categoria: str
- disponible: bool

Métodos:

- obtener_precio()
- actualizar_disponibilidad(disponible: bool)

Relaciones:

- Es clase base para `Bebida` y `Postre`

5. Clase “Bebida” (Hereda de `ProductoBase`)

Atributos:

- tamaño: str
- tipo: str

- cafeina: bool
- opciones_personalizacion: List[str]

Métodos:

- personalizar_bebida(opciones: List[str])
- calcular_precio_final()

Relaciones:

- Hereda de `ProductoBase`
- Puede ser parte de un `Pedido`

6. Clase “Postre” (Hereda de `ProductoBase`)

Atributos:

- vegano: bool
- sin_gluten: bool
- ingredientes: List[str]

Métodos:

- verificar_alergenos()
- obtener_informacion_nutricional()

Relaciones:

- Hereda de `ProductoBase`
- Puede ser parte de un `Pedido`

7. Clase “Inventario”

Atributos:

- ingredientes: Dict[str, int]

- stock_minimo: int

Métodos:

- verificar_stock(ingrediente: str)
- actualizar_cantidad(ingrediente: str, cantidad: int)
- alerta_stock_bajo()

Relaciones:

- Es gestionado por `Empleado`

8. Clase “Pedido”

Atributos:

- numero_pedido: str
- fecha: datetime
- estado: EstadoPedido
- items: List[ProductoBase]
- total: float
- cliente: Cliente

Métodos:

- calcular_total()
- actualizar_estado(nuevo_estado: EstadoPedido)
- agregar_producto(producto: ProductoBase)
- personalizar_producto(producto: Bebida, opciones: List[str])

Relaciones:

- Se relaciona con `Cliente`

- Contiene `ProductoBase` (Bebidas y Postres)
- Es gestionado por `Empleado`

9. Clase “Promoción”

Atributos:

- codigo: str
- descuento: float
- validez_inicio: date
- validez_fin: date
- puntos_requeridos: int

Métodos:

- aplicar_descuento(pedido: Pedido)
- verificar_validez()
- calcular_puntos(total_pedido: float)

Relaciones:

- Se aplica a `Pedido`
- Interactúa con el sistema de puntos de `Cliente`

10. Enum “EstadoPedido”

Estados:

- PENDIENTE
- EN_PREPARACION
- ENTREGADO
- CANCELADO

Relaciones:

- Utilizado por `Pedido` para gestionar estados

Conclusión

Este sistema representa una solución integral para la gestión moderna de restaurantes, abordando las necesidades tanto operativas como de experiencia del cliente. Su diseño modular y las relaciones bien definidas entre clases permiten una fácil extensibilidad y mantenimiento, asegurando que el sistema pueda evolucionar con las necesidades cambiantes del negocio.

Captura del Funcionamiento

```
=== Simulación del Sistema de Restaurante ===

1. Personalización de bebidas:
Precio del Latte personalizado: $5.0

2. Verificación de alérgenos:
Alérgenos en Torta de Chocolate: ['huevos', 'leche']

3. Procesamiento de pedido:
Total del pedido: $8.0
Promoción aplicada. Nuevo total: $6.4

4. Gestión de estados del pedido:
Estado del pedido: en_preparacion

5. Gestión de inventario:
Stock inicial de café: 100
Stock después de preparar el pedido: 98

6. Verificación de stock bajo:
Productos con stock bajo: []

7. Historial de cliente:
Número de pedidos del cliente: 0
...
Nuevo teléfono del cliente: 555-9999

Estado final del pedido: entregado
Puntos de fidelidad actuales: 100
```

Justificación e Importancia del Sistema de gestión de libros (Biblioteca)

JUSTIFICACIÓN DEL SISTEMA

El desarrollo de un sistema de gestión bibliotecaria se justifica por la necesidad de modernizar y optimizar los procesos tradicionales de administración de bibliotecas.

1. Necesidad de Automatización: Los procesos manuales son lentos y propensos a errores humanos, especialmente en bibliotecas con grandes volúmenes de material y múltiples sucursales.

2. Demanda de Servicios Digitales: Los usuarios esperan poder acceder a información sobre disponibilidad de materiales y gestionar sus préstamos de manera digital.

3. Complejidad en la Gestión: La administración de diferentes tipos de materiales (físicos y digitales) requiere un sistema que pueda manejar distintas categorías y formatos.

4. Requisitos de Seguimiento: La necesidad de mantener un registro detallado de préstamos, devoluciones y penalizaciones requiere un sistema automatizado que garantice precisión y consistencia.

5. Gestión Multi-sucursal: La coordinación entre diferentes ubicaciones físicas necesita un sistema centralizado que permita la transferencia y seguimiento de materiales.

IMPORTANCIA DEL SISTEMA

La implementación de este sistema de gestión bibliotecaria tiene una importancia crucial en varios aspectos:

1. Eficiencia Operativa:

- Reduce significativamente el tiempo necesario para procesar préstamos y devoluciones
- Automatiza el cálculo de fechas de vencimiento y penalizaciones
- Minimiza errores en el registro y seguimiento de materiales
- Optimiza la distribución de recursos entre sucursales

2. Mejora en el Servicio al Usuario:

- Permite búsquedas rápidas y precisas en el catálogo
- Facilita el acceso a la información sobre disponibilidad de materiales
- Ofrece transparencia en el proceso de préstamos y penalizaciones
- Proporciona un historial detallado de transacciones

3. Control y Seguimiento:

- Mantiene un registro exacto de la ubicación de cada material
- Permite un seguimiento detallado del historial de préstamos
- Facilita la gestión de inventarios
- Genera datos valiosos para la toma de decisiones

4. Sostenibilidad:

- Reduce el uso de papel en procesos administrativos
- Optimiza la utilización de recursos bibliográficos
- Facilita la preservación y acceso a materiales digitales
- Mejora la durabilidad de los materiales físicos mediante un mejor control

5. Adaptabilidad y Escalabilidad:

- Permite la incorporación de nuevos tipos de materiales
- Facilita la expansión a nuevas sucursales
- Se adapta a cambios en políticas y procedimientos
- Posibilita la integración con otros sistemas

ANÁLISIS DETALLADO DE CLASES

1. Material (Clase Base)

- *Atributos:*

- * id: Identificador único del material
- * titulo: Nombre del material
- * estado: Estado actual (disponible, prestado, en transferencia)
- * ubicacion_actual: Sucursal donde se encuentra el material
- * disponible: Indicador booleano de disponibilidad

- *Métodos:*

- * verificar_disponibilidad(): Comprueba si el material está disponible
- * actualizar_estado(): Modifica el estado actual del material

- *Relaciones:*

- * Clase que hereda a Libro, Revista y MaterialDigital
- * Se relaciona con Sucursal para ubicación
- * Se relaciona con Prestamo

2. Libro (Hereda de Material)

- *Atributos adicionales:*

- * autor: Nombre del autor del libro
- * genero: Categoría o género literario
- * isbn: Número de identificación internacional

- *Relaciones:*

- * Hereda todas las propiedades de Material
- * Puede ser objeto de Prestamo

3. Revista (Hereda de Material)

- *Atributos adicionales:*

- * edicion: Número o fecha de edición

- * periodicidad: Frecuencia de publicación

- * numero: Número de la revista

- *Relaciones:*

- * Hereda todas las propiedades de Material

- * Puede ser objeto de Prestamo

4. MaterialDigital (Hereda de Material)

- *Atributos adicionales:*

- * tipo_archivo: Formato del archivo digital

- * enlace_descarga: URL o ruta de acceso

- * tamano_mb: Tamaño del archivo en megabytes

- *Relaciones:*

- * Hereda todas las propiedades de Material

- * Puede ser objeto de Prestamo

5. Persona (Clase Base)

- *Atributos:*

- * id: Identificador único de la persona

- * nombre: Nombre completo

- * email: Correo electrónico

- * telefono: Número de contacto

- *Relaciones:*

- * Clase padre de Usuario y Bibliotecario

- * Base para el sistema de gestión de personal

6. Usuario (Hereda de Persona)

- Atributos adicionales:

- * historial_prestamos: Lista de préstamos realizados
- * penalizaciones: Registro de penalizaciones
- * multas_pendientes: Monto total de multas sin pagar

- Métodos:

- * consultar_catalogo(): Busca materiales en el catálogo
- * ver_historial_prestamos(): Muestra préstamos anteriores
- * pagar_multa(): Procesa el pago de penalizaciones

- Relaciones:

- * Hereda de Persona
- * Se relaciona con Prestamo y Penalizacion
- * Interactúa con Catalogo

7. Bibliotecario (Hereda de Persona)

- Atributos adicionales:

- * codigo_empleado: Identificador como empleado
- * sucursal_asignada: Sucursal donde trabaja

- Métodos:

- * transferir_material(): Gestiona movimientos entre sucursales
- * aplicar_penalizacion(): Registra multas y sanciones

- Relaciones:

- * Hereda de Persona
- * Se relaciona con Sucursal
- * Gestiona Penalizacion y transferencias

8. Sucursal

- Atributos:

- * id: Identificador único de la sucursal
- * nombre: Nombre de la sucursal
- * direccion: Ubicación física
- * catalogo_local: Lista de materiales disponibles
- * encargado: Bibliotecario responsable

- Métodos:

- * buscar_material(): Localiza materiales en el catálogo local
- * recibir_transferencia(): Procesa llegada de materiales

- Relaciones:

- * Contiene Material
- * Se relaciona con Bibliotecario
- * Forma parte de Catalogo

9. Catalogo

- Atributos:

- * sucursales: Lista de sucursales del sistema

- Métodos:

- * buscar_en_todas_sucursales(): Búsqueda global de materiales

- Relaciones:

- * Agrupa todas las Sucursal
- * Facilita acceso a Material

10. préstamo

- Atributos:

- * id: Identificador único del préstamo
- * usuario: Usuario que realiza el préstamo
- * material: Material prestado
- * fecha_prestamo: Fecha de inicio
- * fecha_devolucion_esperada: Fecha límite
- * fecha_devolucion_real: Fecha real de devolución
- * estado: Estado actual del préstamo

- Métodos:

- * registrar_devolucion(): Procesa la devolución
- * verificar_retraso(): Calcula días de retraso

- Relaciones:

- * Conecta Usuario con Material
- * Puede generar Penalización

11. penalización

- Atributos:

- * id: Identificador único de la penalización
- * usuario: Usuario penalizado
- * monto: Cantidad a pagar
- * motivo: Razón de la penalización
- * fecha: Fecha de aplicación
- * pagada: Estado del pago

- Métodos:

- * registrar_pago(): Procesa el pago de la multa

- *Relaciones:*

- * Se asocia con Usuario

- * Puede ser generada por préstamo

CONCLUSIÓN

El sistema implementa una forma de gestionar los libros de una biblioteca, esto con ayuda de diferentes tipos de clases, haciendo que sea un sistema cuidadosamente estructurado para que la mayoría de las funciones salga como se espera.

Este sistema es de mucha importancia pues hasta genera penalización que como sabemos es muy común en este tipo de lugares pues ya sea por diversas razones se atrasa uno en la entrega de los libros. Ayudando así a mantener un lugar mas eficaz en este caso la biblioteca, además un mejor control en los usuarios.

Captura del sistema

```
=== SISTEMA DE BIBLIOTECA DIGITAL ===

1. Creando sucursales...
2. Inicializando catálogo general...
3. Registrando materiales...
4. Distribuyendo materiales en sucursales...
5. Registrando personal...
6. Registrando usuarios...

=== OPERACIONES DEL SISTEMA ===

7. Búsquedas en el catálogo:

Búsqueda por género 'Ficción':

8. Gestión de préstamos:
Préstamo realizado: 1984 a Juan Pérez
Préstamo realizado (simulado con retraso): Cien años de soledad a Laura Sánchez

9. Procesando devoluciones y penalizaciones:
Devolución normal: 1984
Devolución con retraso: Cien años de soledad
Penalización aplicada: $6.0 por 6 días de retraso
...
- Curso Python: disponible
  Ubicación: Biblioteca Central
- Revista Digital: disponible
  Ubicación: Biblioteca Sur
```